

MF 731 Homework 1: Volatility of SPY

Leonid Zhuravlev

September 21, 2026

Code: <https://github.com/zhurleonid/corporate-risk-management> (`homework_1/hw1_code.ipynb`)

The write-up is below. The code is appended at the end.

1 Setup

Daily SPY closes from Yahoo Finance. Log returns are $x_i = \log(s_i/s_{i-1})$, and I take $\Delta = 1/252$ throughout, so annualized volatility is $\hat{\sigma}_A = \sqrt{\hat{\sigma}^2/\Delta}$.

The download starts in June 2020, not August 2021. The extra year is there so the 100-day window is already full on the first day I have to plot. That leaves 1552 closes and 1551 returns.

2 MA and EWMA estimates

2.1 Pseudocode

```
read closes s_0 ... s_N, set n = 100, lambda = 0.94, Delta = 1/252
x_i = log(s_i / s_{i-1})          for i = 1 ... N

MA(n):
for i = n+1 ... N:
    window = x_{i-n} ... x_{i-1}    the n returns ending the day before
    mu      = mean(window)
    var_i   = sum over window of (x - mu)^2 / (n - 1)

EWMA(lambda):
var = variance of x_1 ... x_n      (same (n-1) divisor)
for i = n+1 ... N:
    var = lambda*var + (1 - lambda)*x_{i-1}^2
    var_i = var

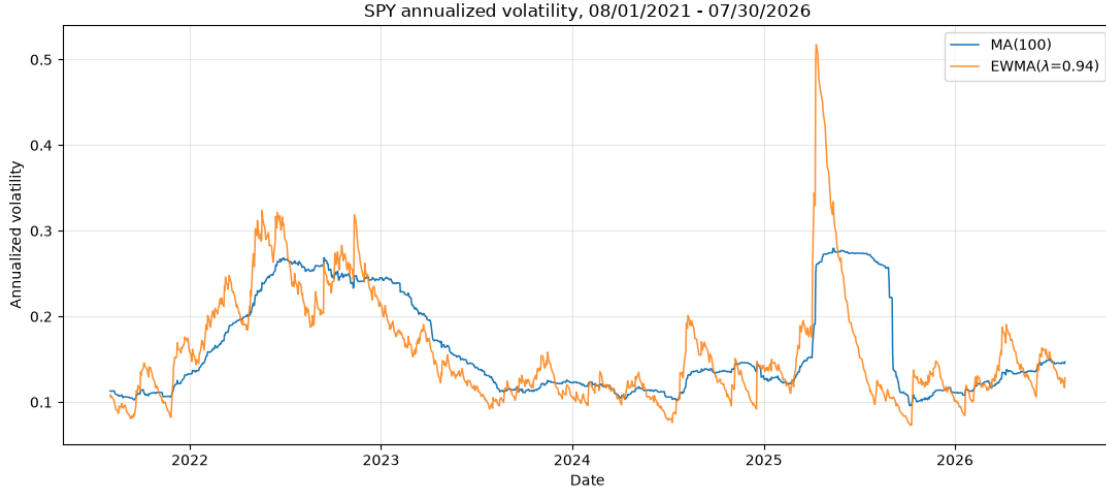
annualize both:  sigma_i = sqrt(var_i / Delta)
plot MA and EWMA on the same axes, restricted to 08/01/2021 - 07/30/2026
```

Neither estimator looks ahead: day i uses returns through $i - 1$ only, so $\sigma_{t+\Delta}$ is known at t .

The problem lets me seed EWMA with the sample variance of the past 100 days. I seed at the start of the download, which gives the recursion 194 steps to settle before the plot window opens. Under the alternative reading, where the seed comes from the 100 days right before 08/01/2021,

the first plotted value moves by 0.0027, and nothing after day 51 moves by more than 10^{-4} . Mean, min and max come out the same to four decimals.

2.2 Results



Annualized volatility of SPY daily log returns, 08/01/2021-07/30/2026, estimated by MA(100) and EWMA(lambda = 0.94).

	MA(100)	EWMA(0.94)
mean	0.1614	0.1583
std	0.0576	0.0664
min	0.0958	0.0730
max	0.2794	0.5177

Both series average about the same, 16.1% against 15.8%, and there the similarity ends. EWMA is far more reactive. It peaks at 51.8% against the MA's 27.9%, and it comes back down almost as quickly as it went up.

The MA is smoother only in the sense that it moves less day to day. What it actually does is hold every shock for exactly 100 days and then drop it. The April 2025 spike shows this plainly: the MA climbs, sits near its peak for months, then falls off a cliff the day those returns leave the window. The plateaus and cliffs in the plot are all this same artifact.

3 Fitting GARCH(1,1)

The model is

$$\sigma_{t+\Delta}^2 = \alpha_0 + \alpha_1 X_t^2 + \beta_1 \sigma_t^2, \quad X_{t+\Delta} = \sigma_{t+\Delta} Z_{k+1}, \quad Z \sim N(0, 1).$$

With Gaussian Z , maximizing the log-likelihood reduces to minimizing

$$\sum_i \left(\log \sigma_i^2 + \frac{x_i^2}{\sigma_i^2} \right), \quad \sigma_i^2 = \alpha_0 + \alpha_1 x_{i-1}^2 + \beta_1 \sigma_{i-1}^2.$$

3.1 Pseudocode

```

x = the returns from 06/01/2025 to 05/31/2026          (250 of them)

function neg_loglik(a0, a1, b1):
    if a0 <= 0 or a1 < 0 or b1 < 0 or a1 + b1 >= 1:
        return a huge number          keeps the search inside the region
    var = variance(x)                  sigma_1^2
    total = 0
    for t = 1 ... 250:
        total = total + log(var) + x_t^2 / var
        var = a0 + a1*x_t^2 + b1*var    variance for the NEXT day
    return total

(a0, a1, b1) = the (a0,a1,b1) minimizing neg_loglik, by Nelder-Mead
               from 3 starting guesses, keeping whichever lands lowest
V_L = a0 / (1 - a1 - b1)

in-sample fitted volatility, same recursion, parameters now fixed:
    var = variance(x)
    for t = 1 ... 250:
        record sqrt(var / Delta)
        var = a0 + a1*x_t^2 + b1*var
    keep the final var as sigma^2_N for reference

```

3.2 Results

The fit window holds 250 trading days, 06/02/2025 through 05/29/2026. Forming that first return needs the 05/30/2025 close, one price outside the stated window. That is unavoidable when the window is given in prices and the model is fitted on returns.

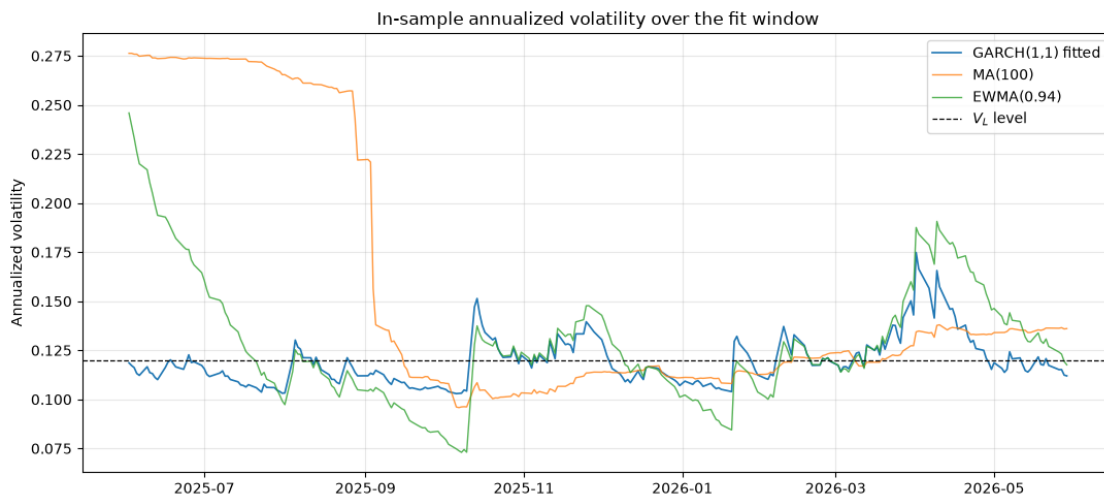
Minimizing the negative log-likelihood with Nelder-Mead:

α_0	6.7425×10^{-6}
α_1	0.0581
β_1	0.8236
$\alpha_1 + \beta_1$	0.8817
V_L	5.6987×10^{-5}
$\sqrt{V_L}/\Delta$ (ann. vol)	11.98%
$-\ell(\hat{\theta})$	-2199.64

With $\alpha_1 + \beta_1 = 0.8817 < 1$ the fit is stationary, so forecasts pull back toward V_L . The stationarity constraint was not binding at the optimum. Since β_1 dwarfs α_1 , a shock enters the variance weakly on the day it lands and then decays.

How fast it decays is worth dwelling on. $(\alpha_1 + \beta_1)^n$ halves in $\log 0.5 / \log 0.8817 = 5.5$ days and is 90% gone inside 18. Multi-year fits of daily SPY typically land near 0.98, a half-life closer to 34 days. This window, a calm year apart from March, mean-reverts unusually quickly. That is exactly why the forecasts below flatten onto V_L within about three weeks.

I ran the optimizer from three different starting guesses and they all converged to the same parameters, so this appears to be the actual minimum rather than a local one.



In-sample annualized volatility over the fit window: GARCH(1,1) conditional volatility against MA(100), EWMA(0.94), and the long-run level.

4 Forecasting 06/01/2026 - 07/30/2026

The notes' pseudocode sets σ_N^2 to the sample variance of the fit returns, 11.86% annualized. From there I roll the recursion forward, drawing a fresh shock each day, and repeat $M = 100$ times over the $m = 42$ trading days in the horizon.

Carrying the filtered σ_N^2 out of the fit would be cleaner in principle, since it is the model's own state at the forecast origin, 11.01% here. I follow the notes instead. Given how fast this fit mean-reverts, the two seeds converge within days.

4.1 Pseudocode

```
use the fitted a0, a1, b1 and V_L from part 2
var_start = variance(x)           the notes' choice of sigma_N^2
x_last    = the last return of the fit window
m = 42 days from 06/01/2026 to 07/30/2026, M = 100

simulated paths:
for j = 1 ... M:
    var    = var_start
    x_prev = x_last
    for d = 1 ... m:
```

```

var          = a0 + a1*x_prev^2 + b1*var
path[j][d]   = sqrt(var / Delta)
draw Z from N(0,1)
x_prev       = sqrt(var)*Z           the simulated next return

```

expected path:

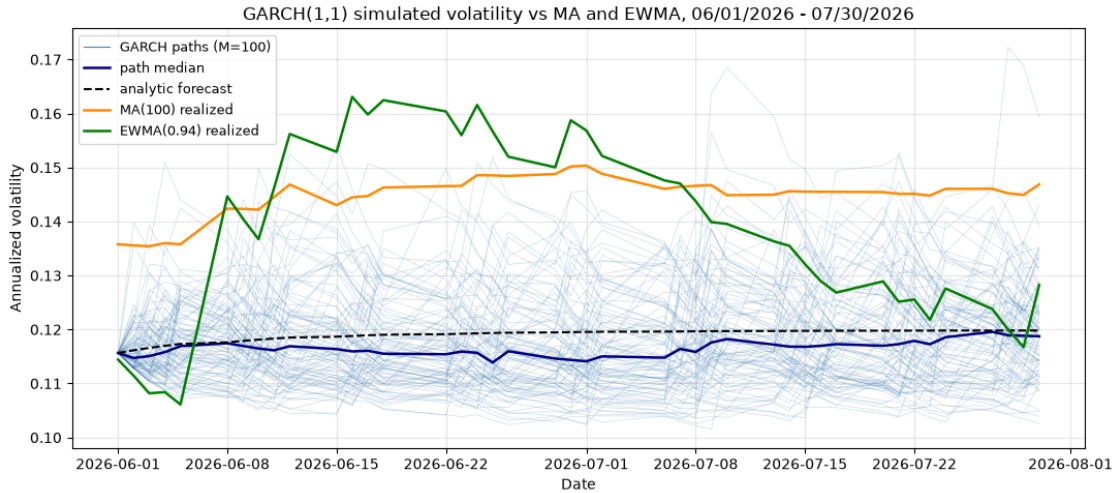
```

sigma^2_1 = a0 + a1*x_last^2 + b1*var_start
           this is the d = 1 value of the loop above, identical for every
           path, which is why day 1 has no spread
for n = 1 ... m:
    E_n = V_L + (a1 + b1)^(n-1) * (sigma^2_1 - V_L)
    record sqrt(E_n / Delta)

```

take the 5%, 50% and 95% quantiles across the M paths for each day
plot the paths, their median, the expected path, and the MA and EWMA
estimates over the same 42 days

4.2 Results



100 simulated GARCH(1,1) annualized volatility paths over 06/01/2026-07/30/2026, with the path median and the analytic expected path, compared against the realized MA(100) and EWMA(0.94) estimates.

Date	GARCH med.	5%	95%	analytic	MA(100)	EWMA(0.94)
2026-06-01	0.1156	0.1156	0.1156	0.1156	0.1358	0.1144
2026-06-12	0.1169	0.1072	0.1367	0.1185	0.1468	0.1562
2026-06-29	0.1146	0.1051	0.1352	0.1195	0.1488	0.1500
2026-07-14	0.1168	0.1063	0.1399	0.1197	0.1456	0.1355
2026-07-30	0.1187	0.1088	0.1350	0.1198	0.1469	0.1282

4.3 Comparison

Every path starts from the same variance, so they sit tight for the first few days and fan out as shocks accumulate. The median and the analytic expectation

$$\mathbb{E}[\sigma_{t+n\Delta}^2 \mid \mathcal{F}_t] = V_L + (\alpha_1 + \beta_1)^{n-1} (\sigma_{t+\Delta}^2 - V_L)$$

both start at $\sigma_{N+1}^2 = 11.56\%$ annualized, identical for every path, which is why day 1 shows no spread at all, and drift up to roughly 12.0% by the end. EWMA cannot produce this shape: its forecast is flat by construction, the criticism the notes raise.

Set against the realized estimates, EWMA lands inside the 5-95% band on 35.7% of days. The MA sits above it on all 42.

It would be tempting to explain that gap by saying the GARCH starts from a quieter stretch, but the numbers do not support it. The seed is the sample variance of the whole fit year, 11.86% annualized, and that already contains March 2026 at 18.2%. The filtered alternative would start lower still, at 11.01%. Neither matters much: with $\alpha_1 + \beta_1 = 0.8817$ the forecast is pinned near $V_L \approx 12\%$ within days regardless of where it begins.

The real difference is that the MA never reverts to anything. Its estimate is the average squared return over the previous 100 days, full stop. On 06/01/2026 that average still includes March, and it keeps including March for the whole horizon, which is why the line sits flat near 14.5% until those returns finally roll out. What we are seeing is the MA's look-back, not the GARCH's starting point.

Realized volatility over the horizon came in at 15.0% annualized. The GARCH median was 11.7%, and the fit year itself realized 11.9%. So the model underforecast, and it is worth being clear about why: nothing in a calm 250-day sample points to the June move. A mean-reverting model pulled toward V_L cannot anticipate a change of regime, which is the “did we see COVID coming?” example from lecture. This is the argument for short projection horizons, and for refitting as the environment moves.

5 Appendix: source code

The code below produces every figure and number above.

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import minimize

DELTA = 1 / 252
N_MA = 100
LAM = 0.94

plt.rcParams['figure.figsize'] = (11, 5)
```

```
[2]: # SPY daily closes. Saved to csv so this runs offline and stays reproducible.
# To pull fresh data instead:
#     import yfinance as yf
#     prices = yf.download('SPY', start='2020-06-01', end='2026-08-05')['Close']
# Starts June 2020 so the 100-day window is full by 08/01/2021.
prices = pd.read_csv('spy_daily.csv', parse_dates=['Date']).
    ↪set_index('Date')['Close']
x = np.log(prices / prices.shift(1)).dropna()

print(prices.index[0].date(), '->', prices.index[-1].date())
print('trading days:', len(prices), ' log returns:', len(x))
```

2020-06-01 -> 2026-08-04

trading days: 1552 log returns: 1551

```
[3]: def ma_variance(x, n):
    # shift so day i uses the n returns ending at i-1
    return x.rolling(n).var(ddof=1).shift(1)

def ewma_variance(x, lam, n_init):
    var = np.empty(len(x))
    var[:] = np.nan
    v = x.iloc[:n_init].var(ddof=1)      # init from the first 100 returns
    for i in range(n_init, len(x)):
        v = lam * v + (1 - lam) * x.iloc[i - 1] ** 2
        var[i] = v
    return pd.Series(var, index=x.index)

def annualize(var):
    return np.sqrt(var / DELTA)

ma_vol = annualize(ma_variance(x, N_MA))
ewma_vol = annualize(ewma_variance(x, LAM, N_MA))
```

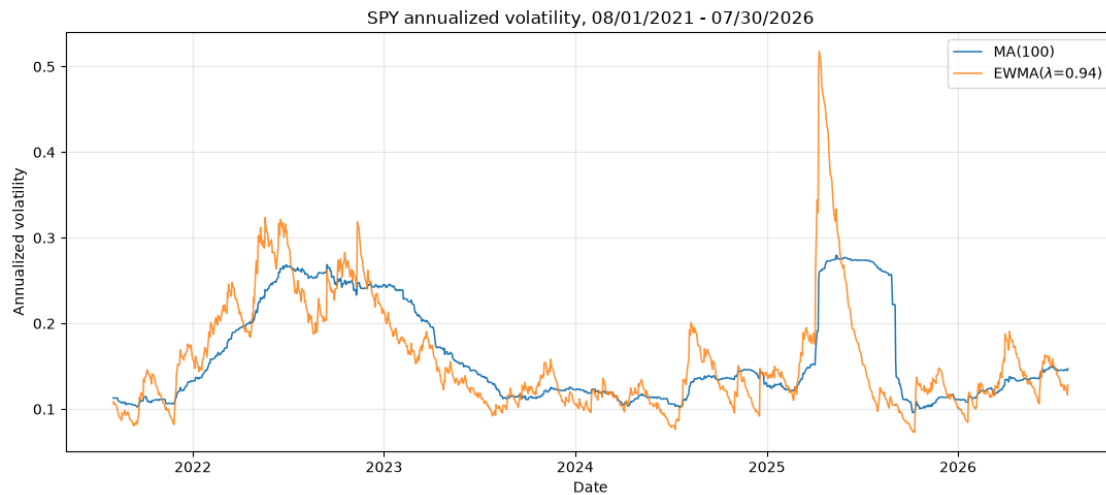
```
[4]: start, end = '2021-08-01', '2026-07-30'
ma_w = ma_vol.loc[start:end]
ewma_w = ewma_vol.loc[start:end]

fig, ax = plt.subplots()
ax.plot(ma_w.index, ma_w.values, label=f'MA({N_MA})', lw=1.1)
ax.plot(ewma_w.index, ewma_w.values, label=rf'EWMA($\lambda$={LAM})', lw=1.1,
    ↪alpha=0.85)
ax.set_title('SPY annualized volatility, 08/01/2021 - 07/30/2026')
ax.set_xlabel('Date')
```

```

ax.set_ylabel('Annualized volatility')
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```



```

[5]: summary = pd.DataFrame({'MA(100)': ma_w, f'EWMA({LAM})': ewma_w})
summary.describe().loc[['mean', 'std', 'min', 'max']]

```

```

[5]:      MA(100)  EWMA(0.94)
mean  0.161420  0.158337
std    0.057604  0.066392
min    0.095806  0.073006
max    0.279434  0.517705

```

```

[6]: fit_returns = x.loc['2025-06-01':'2026-05-31']
xf = fit_returns.values
print('fit window:', fit_returns.index[0].date(), '->', fit_returns.index[-1].
      ↪date(),
      ' n =', len(xf))

```

fit window: 2025-06-02 -> 2026-05-29 n = 250

```

[7]: def neg_log_lik(params, r):
      # -loglik up to a constant, with sigma^2 built by the GARCH recursion
      a0, a1, b1 = params
      if a0 <= 0 or a1 < 0 or b1 < 0 or a1 + b1 >= 1:
          return 1e10
      var = r.var()          # sigma_1^2
      total = 0.0

```

```

for t in range(len(r)):
    total += np.log(var) + r[t] ** 2 / var
    var = a0 + a1 * r[t] ** 2 + b1 * var
return total

def fit_garch(r):
    # a few starting points, keep the best - guards against a local min
    guesses = [(1e-6, 0.10, 0.85), (5e-6, 0.05, 0.90), (2e-6, 0.20, 0.70)]
    best = None
    for g in guesses:
        res = minimize(neg_log_lik, g, args=(r,), method='Nelder-Mead',
                      options={'maxiter': 8000, 'xatol': 1e-12, 'fatol': 1e-10})
        if best is None or res.fun < best.fun:
            best = res
    return best

res = fit_garch(xf)
a0, a1, b1 = res.x
VL = a0 / (1 - a1 - b1)

print(f'alpha_0 = {a0:.4e}')
print(f'alpha_1 = {a1:.4f}')
print(f'beta_1 = {b1:.4f}')
print(f'alpha_1 + beta_1 = {a1 + b1:.4f}')
print(f'V_L = {VL:.4e}    -> long-run annualized vol = {np.sqrt(VL / DELTA):.2%}')
print(f'negative log-likelihood = {res.fun:.4f}')

```

```

alpha_0 = 6.7425e-06
alpha_1 = 0.0581
beta_1 = 0.8236
alpha_1 + beta_1 = 0.8817
V_L = 5.6987e-05    -> long-run annualized vol = 11.98%
negative log-likelihood = -2199.6403

```

```

[8]: # in-sample vol from the fitted params
def garch_filter(r, a0, a1, b1, var0=None):
    var = np.empty(len(r))
    v = r.var() if var0 is None else var0
    for t in range(len(r)):
        var[t] = v
        v = a0 + a1 * r[t] ** 2 + b1 * v
    return var, v

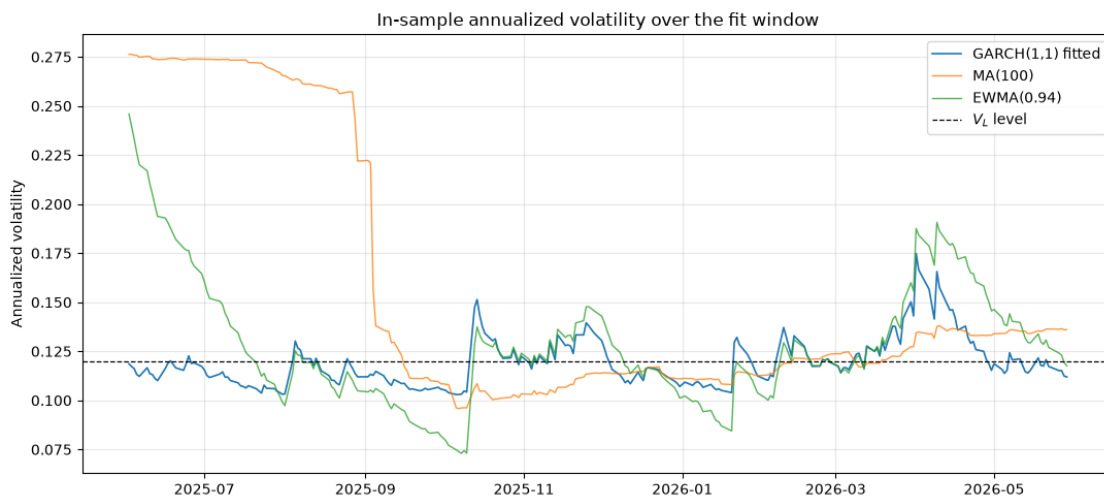
```

```

in_var, var_next = garch_filter(xf, a0, a1, b1)
in_vol = pd.Series(np.sqrt(in_var / DELTA), index=fit_returns.index)

fig, ax = plt.subplots()
ax.plot(in_vol.index, in_vol.values, label='GARCH(1,1) fitted', lw=1.2)
ax.plot(ma_vol.loc[fit_returns.index[0]:fit_returns.index[-1]],
        label='MA(100)', lw=1.0, alpha=0.8)
ax.plot(ewma_vol.loc[fit_returns.index[0]:fit_returns.index[-1]], label='EWMA(0.
        94)', lw=1.0, alpha=0.8)
ax.axhline(np.sqrt(VL / DELTA), color='k', ls='--', lw=0.9, label='$V_L$ level')
ax.set_title('In-sample annualized volatility over the fit window')
ax.set_ylabel('Annualized volatility')
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```



```

[9]: forecast_dates = x.loc['2026-06-01':'2026-07-30'].index
m = len(forecast_dates)
M = 100
print('forecast horizon:', forecast_dates[0].date(), '->', forecast_dates[-1].
      date(), ' m =', m)

```

forecast horizon: 2026-06-01 -> 2026-07-30 m = 42

```

[10]: rng = np.random.default_rng(42)

# initial variance = sample variance of the fit-window returns
var_start = xf.var()

```

```

paths = np.empty((M, m))
for j in range(M):
    var = var_start
    x_prev = xf[-1]
    for d in range(m):
        var = a0 + a1 * x_prev ** 2 + b1 * var
        paths[j, d] = np.sqrt(var / DELTA)
        x_prev = np.sqrt(var) * rng.standard_normal()

paths = pd.DataFrame(paths.T, index=forecast_dates)
paths.shape

```

[10]: (42, 100)

```

[11]: # expected path. The notes give
#  $E[\sigma^2_{\{t+n\Delta\}} | F_t] = V_L + (a_1+b_1)^{n-1} (\sigma^2_{\{t+\Delta\}} - V_L)$ 
# so the base is the ONE-STEP-AHEAD variance, not the starting variance, and the
# exponent is n-1. Day 1 is then  $\sigma^2_{\{N+1\}}$ , which is deterministic - the
↳ same
# value every path starts from.
var1 = a0 + a1 * xf[-1] ** 2 + b1 * var_start #  $\sigma^2_{\{N+1\}}$ 
k = np.arange(1, m + 1)
exp_var = VL + (a1 + b1) ** (k - 1) * (var1 - VL)
exp_vol = pd.Series(np.sqrt(exp_var / DELTA), index=forecast_dates)

ma_f = ma_vol.loc[forecast_dates]
ewma_f = ewma_vol.loc[forecast_dates]

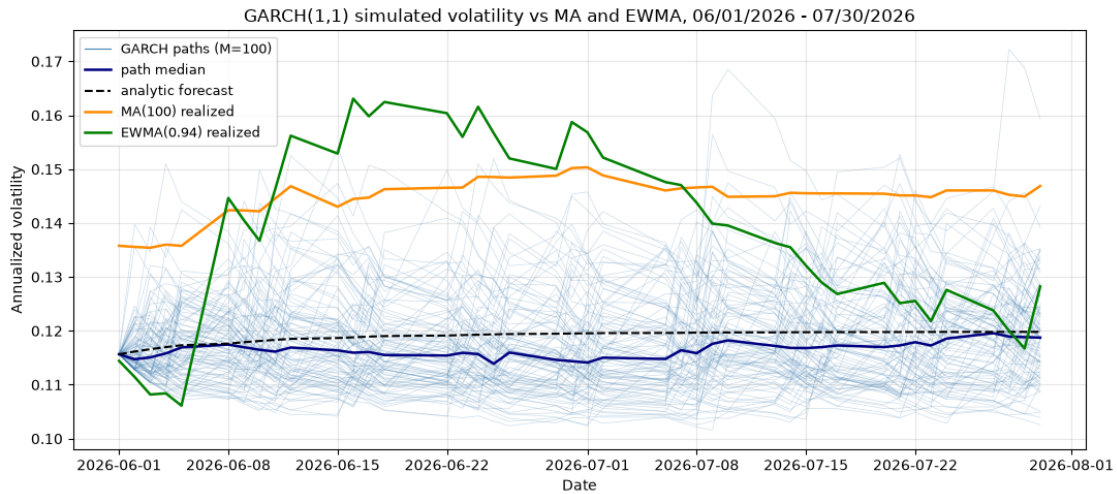
```

```

[12]: fig, ax = plt.subplots()
ax.plot(paths.index, paths.values, color='steelblue', lw=0.5, alpha=0.25)
ax.plot([], [], color='steelblue', lw=0.8, label=f'GARCH paths (M={M})')
ax.plot(paths.index, paths.median(axis=1), color='navy', lw=1.8, label='path
↳ median')
ax.plot(exp_vol.index, exp_vol.values, color='k', ls='--', lw=1.5,
↳ label='analytic forecast')
ax.plot(ma_f.index, ma_f.values, color='darkorange', lw=1.8, label='MA(100)
↳ realized')
ax.plot(ewma_f.index, ewma_f.values, color='green', lw=1.8, label='EWMA(0.94)
↳ realized')
ax.set_title('GARCH(1,1) simulated volatility vs MA and EWMA, 06/01/2026 - 07/
↳ 30/2026')
ax.set_xlabel('Date')
ax.set_ylabel('Annualized volatility')
ax.legend(fontsize=9)
ax.grid(alpha=0.3)
plt.tight_layout()

```

```
plt.show()
```



```
[13]: q = paths.quantile([0.05, 0.5, 0.95], axis=1).T
comp = pd.DataFrame({
    'GARCH median': q[0.5],
    'GARCH 5%': q[0.05],
    'GARCH 95%': q[0.95],
    'analytic': exp_vol,
    'MA(100)': ma_f,
    'EWMA(0.94)': ewma_f,
})
comp.iloc[[0, 9, 19, 29, m - 1]].round(4)
```

```
[13]:
```

	GARCH median	GARCH 5%	GARCH 95%	analytic	MA(100)	EWMA(0.94)
Date						
2026-06-01	0.1156	0.1156	0.1156	0.1156	0.1358	0.1144
2026-06-12	0.1169	0.1072	0.1367	0.1185	0.1468	0.1562
2026-06-29	0.1146	0.1051	0.1352	0.1195	0.1488	0.1500
2026-07-14	0.1168	0.1063	0.1399	0.1197	0.1456	0.1355
2026-07-30	0.1187	0.1088	0.1350	0.1198	0.1469	0.1282

```
[14]: inside = ((ma_f.values >= q[0.05].values) & (ma_f.values <= q[0.95].values)).
    ↪mean()
inside_e = ((ewma_f.values >= q[0.05].values) & (ewma_f.values <= q[0.95].
    ↪values)).mean()
print(f'MA(100) inside the 5-95% GARCH band: {inside:.1%} of days')
print(f'EWMA(0.94) inside the 5-95% GARCH band: {inside_e:.1%} of days')
print()
print(f'mean over horizon    GARCH median {q[0.5].mean():.4f}    '
      f'MA {ma_f.mean():.4f}    EWMA {ewma_f.mean():.4f}')
```

```

print(f'long-run V_L vol = {np.sqrt(VL / DELTA):.4f}')
print(f'mean-reversion half-life = {np.log(0.5) / np.log(a1 + b1):.1f} days')

realized = x.loc[forecast_dates].std(ddof=1) / np.sqrt(DELTA)
fit_realized = fit_returns.std(ddof=1) / np.sqrt(DELTA)
print()
print(f'realized ann vol over fit window:      {fit_realized:.4f}')
print(f'realized ann vol over forecast window: {realized:.4f}')

```

MA(100) inside the 5-95% GARCH band: 0.0% of days
 EWMA(0.94) inside the 5-95% GARCH band: 35.7% of days

mean over horizon GARCH median 0.1165 MA 0.1447 EWMA 0.1384
 long-run V_L vol = 0.1198
 mean-reversion half-life = 5.5 days

realized ann vol over fit window: 0.1188
 realized ann vol over forecast window: 0.1500